

CPU, Memory & Storage Compression Implementation Guide

Dimensional Folding for Container-Based Infrastructure

Christian Kilpatrick
Advanced Research Array
ck@advancedresearcharray.com
ORCID: 0009-0003-2275-8410

February 2026

Abstract

This implementation guide covers the deployment of the Array Folding Compression Engine for reducing CPU utilization, memory footprint, and storage consumption within container-based infrastructure. The three-stage compression pipeline—dimensional folding, BLSB bit-level quantization, and entropy coding—reduces the working memory required for high-dimensional data by up to 48×, cuts storage requirements proportionally, and lowers CPU cycles spent on search, comparison, and transfer operations over that data. All operations run exclusively inside containers. This guide targets Debian Linux containers (LXC, Docker). Rocky Linux container support is under active testing and is not yet validated for production use.

Platform Support

Platform	Status	Notes
Debian 11/12 (LXC, Docker)	Production Ready	Fully tested
Rocky Linux 9 (LXC, Docker)	Validated	71/71 tests passing (glibc 2.34)
Rocky Linux 8 (LXC, Docker)	Under Testing	glibc 2.28; not yet validated

All workloads run **inside containers only**. Do not run on the host/node directly.

Contents

1	Introduction	3
2	Impact Summary	3
3	Prerequisites	3
4	Container Setup	4
4.1	Debian LXC Container (Proxmox)	4
4.2	Docker Container	4
4.3	Install Inside the Container	4
5	Memory Compression	4
5.1	Basic Memory Reduction	4
5.2	Container Memory Allocation	5
5.3	Fold on Ingest	5

6	CPU Compression	5
6.1	Search Operations	5
6.2	Batch Processing	6
7	Storage Optimization	6
7.1	Level 1: Dimensional Folding Only	6
7.2	Level 2: Fold + Quantize	7
7.3	Level 3: Fold + Quantize + Compress (Maximum)	7
7.4	Storage Impact Table	7
8	Deploying the Compression Engine in a Container	8
8.1	Service Setup	8
8.2	Systemd Service (Inside Container)	8
8.3	API Endpoints	8
9	Rocky Linux Container Support	9
9.1	Rocky Linux 9 — Validated	9
9.2	Rocky Linux 8 — Under Testing	10
10	Testing Checklist	10
11	Disclaimer	10

1 Introduction

When applications work with high-dimensional data—vector embeddings, feature arrays, sensor readings, model weights—the data’s dimensionality directly drives three infrastructure costs:

1. **Memory.** Each 768-dimension float64 vector occupies 6,144 bytes of RAM. One million vectors requires 5.9 GB. After folding to 16 dimensions, the same million vectors fit in 122 MB—a 48× reduction in container memory allocation.
2. **CPU.** Distance calculations, nearest-neighbor search, and comparison operations scale with dimensionality. Reducing 768D to 16D means each operation touches 48× fewer values, directly reducing CPU cycles.
3. **Storage.** On-disk representation shrinks proportionally. Combined with BLSB quantization (64-bit floats to 16-bit integers) and entropy coding, storage reductions of 72× or more are achievable.

This guide walks through deploying these optimizations inside your containers.

2 Impact Summary

Resource	Before	After (16D)	Reduction
<i>Per 1M vectors at 768D</i>			
Container RAM	5.9 GB	122 MB	48×
Search CPU/query	768 FLOPs/vec	16 FLOPs/vec	48×
Disk storage	5.9 GB	~82 MB	72× (with quant+zlib)
<i>Per 1M vectors at 384D</i>			
Container RAM	2.9 GB	122 MB	24×
Search CPU/query	384 FLOPs/vec	16 FLOPs/vec	24×
Disk storage	2.9 GB	~82 MB	36×
<i>Per 1M vectors at 128D</i>			
Container RAM	976 MB	122 MB	8×
Search CPU/query	128 FLOPs/vec	16 FLOPs/vec	8×
Disk storage	976 MB	~82 MB	12×

Table 1: Resource impact by source dimensionality, folded to 16D. Preservation: 99.7%.

3 Prerequisites

- A **Debian 11 or 12** container (LXC or Docker). Rocky Linux support is under testing—do not use for production yet.
- Python 3.9+ with pip (installed inside the container)
- Array Folding SDK with a valid license key
- For native C acceleration: `libarray_core.so` (x86_64, glibc 2.31+)
- Minimum container resources: 1 vCPU, 512 MB RAM, 20 GB disk (for testing; production sizing depends on dataset)

4 Container Setup

All commands below are executed inside the container.

4.1 Debian LXC Container (Proxmox)

```
1 # On the Proxmox host: create the container
2 pct create 300 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
3   --hostname array-compress \
4   --cores 1 --memory 512 --swap 256 \
5   --rootfs local-lvm:20 \
6   --net0 name=eth0,bridge=vbr0,ip=dhcp
7 pct start 300
8
9 # Push the SDK files into the container
10 pct push 300 arrayfolding-1.0.0-py3-none-any.whl /opt/arrayfolding/
11 pct push 300 libarray_core.so /opt/arrayfolding/
12
13 # Enter the container
14 pct enter 300
```

4.2 Docker Container

```
1 docker run -d --name array-compress \
2   -v /path/to/sdk:/opt/arrayfolding \
3   debian:12 sleep infinity
4
5 docker exec -it array-compress bash
```

4.3 Install Inside the Container

```
1 apt update && apt install -y python3 python3-pip
2 pip install /opt/arrayfolding/arrayfolding-1.0.0-py3-none-any.whl \
3   --break-system-packages
4 export ARRAY_NATIVE_LIB=/opt/arrayfolding/libarray_core.so
```

5 Memory Compression

The primary memory optimization: fold high-dimensional data in-memory so the container requires less RAM.

5.1 Basic Memory Reduction

```
1 import arrayfolding as af
2 import numpy as np
3 import sys
4
5 af.activate("ARRAY-XXXXX-XXXXX-...")
6
7 # Simulate 1M embeddings at 768D
8 embeddings = np.random.randn(1_000_000, 768)
```

```

9 print(f"Original memory: {embeddings.nbytes/1e9:.2f} GB")
10 # Original memory: 5.90 GB
11
12 # Fold to 16D in the container
13 folded = af.batch_fold(embeddings, source_dims=768, target_dims=16)
14 print(f"Folded memory: {folded.nbytes/1e9:.3f} GB")
15 # Folded memory: 0.122 GB
16
17 print(f"RAM reduction: {embeddings.nbytes/folded.nbytes:.0f}x")
18 # RAM reduction: 48x
19
20 # Free the originals -- the container now uses 48x less RAM
21 del embeddings

```

5.2 Container Memory Allocation

After folding, you can reduce the container's memory allocation:

```

1 # Before folding: container needs 8 GB for 1M x 768D + overhead
2 pct set 300 --memory 8192
3
4 # After folding: container only needs ~512 MB for 1M x 16D +
  overhead
5 pct set 300 --memory 512

```

5.3 Fold on Ingest

For the best memory savings, fold data as it enters the container rather than loading full-dimensional data first:

```

1 import arrayfolding as af
2 import numpy as np
3
4 af.activate("ARRAY-XXXXX-...")
5
6 # Process in chunks to keep peak memory low
7 CHUNK_SIZE = 10_000
8 folded_chunks = []
9
10 for chunk in load_embeddings_in_chunks(CHUNK_SIZE):
11     # chunk is (10000, 768) -- fold immediately
12     folded = af.batch_fold(chunk, source_dims=768, target_dims=16)
13     folded_chunks.append(folded)
14     # chunk goes out of scope, memory freed
15
16 # Final array: (N, 16) instead of (N, 768)
17 all_folded = np.vstack(folded_chunks)

```

6 CPU Compression

Folding reduces CPU cost for every operation that touches the data.

6.1 Search Operations

```

1 import arrayfolding as af
2 import numpy as np
3 import time
4
5 af.activate("ARRAY-XXXXX-...")
6
7 n_vectors = 500_000
8 dims_768 = np.random.randn(n_vectors, 768)
9 dims_16 = af.batch_fold(dims_768, source_dims=768, target_dims=16)
10
11 query_768 = np.random.randn(768)
12 query_16 = af.fold(query_768, source_dims=768, target_dims=16)
13
14 # Brute-force search at 768D
15 t0 = time.perf_counter()
16 dists_768 = np.linalg.norm(dims_768 - query_768, axis=1)
17 top10_768 = np.argsort(dists_768)[:10]
18 time_768 = time.perf_counter() - t0
19
20 # Same search at 16D
21 t0 = time.perf_counter()
22 dists_16 = np.linalg.norm(dims_16 - query_16, axis=1)
23 top10_16 = np.argsort(dists_16)[:10]
24 time_16 = time.perf_counter() - t0
25
26 print(f"768D search: {time_768*1000:.1f}ms")
27 print(f"16D search: {time_16*1000:.1f}ms")
28 print(f"CPU speedup: {time_768/time_16:.0f}x")

```

6.2 Batch Processing

Any operation that iterates over dimensions benefits: aggregation, distance matrices, clustering, classification feature extraction. The CPU cost scales linearly with dimensionality, so $768 \rightarrow 16$ reduces CPU time by $\sim 48\times$ per operation.

7 Storage Optimization

Three levels of storage reduction, each building on the previous:

7.1 Level 1: Dimensional Folding Only

```

1 import arrayfolding as af
2 import numpy as np
3
4 data = np.random.randn(1_000_000, 768)
5 folded = af.batch_fold(data, source_dims=768, target_dims=16)
6
7 # Save folded data
8 np.save("/data/embeddings_folded.npy", folded)
9
10 # Disk: 768D = 5.9 GB, 16D = 122 MB (48x reduction)

```

7.2 Level 2: Fold + Quantize

Reduce 64-bit floats to 16-bit integers for additional 4× savings:

```
1 import arrayfolding as af
2 import numpy as np
3
4 folded = af.batch_fold(data, source_dims=768, target_dims=16)
5
6 # Quantize to 16-bit
7 vmin, vmax = folded.min(), folded.max()
8 scale = 65535.0 / max(vmax - vmin, 1e-15)
9 quantized = ((folded - vmin) * scale).astype(np.uint16)
10
11 np.save("/data/embeddings_quant.npy", quantized)
12 # Also save dequantization params
13 np.savez("/data/embeddings_meta.npy", vmin=vmin, vmax=vmax)
14
15 # Disk: 16D x uint16 = 30.5 MB (193x reduction from original)
```

7.3 Level 3: Fold + Quantize + Compress (Maximum)

For archival or cold storage, add AFLD compression:

```
1 import arrayfolding as af
2 import numpy as np
3
4 result = af.compress_vectors(
5     data,
6     source_dims=768,
7     target_dims=16,
8     precision_bits=16
9 )
10
11 with open("/data/embeddings.afld", "wb") as f:
12     f.write(result['compressed'])
13
14 print(f"Original: {result['original_bytes']:>12,} bytes")
15 print(f"Compressed: {result['compressed_bytes']:>12,} bytes")
16 print(f"Ratio: {result['total_ratio']:.0f}x")
17
18 # Disk: ~82 MB compressed (72x reduction from original)
```

7.4 Storage Impact Table

Method	1M vecs @ 768D	1M vecs @ 384D	Reduction
Original (float64)	5,900 MB	2,900 MB	1×
Fold only (16D, float64)	122 MB	122 MB	24–48×
Fold + quantize (uint16)	30.5 MB	30.5 MB	95–193×
Fold + quantize + compress	~15–20 MB	~15–20 MB	145–390×

Table 2: Container disk savings by compression level.

8 Deploying the Compression Engine in a Container

For applications that need compression as a service (e.g., compressing data between containers), deploy the engine as a FastAPI service inside a dedicated container.

8.1 Service Setup

```
1 # Inside the container:
2 pip install fastapi uvicorn numpy --break-system-packages
3
4 export ARRAY_LICENSE_KEY="ARRAY-XXXXX-..."
5 export ARRAY_NATIVE_LIB=/opt/arrayfolding/libarray_core.so
6
7 # Start the compression service
8 uvicorn server:app --host 0.0.0.0 --port 8200 --workers 2
```

8.2 Systemd Service (Inside Container)

Create `/etc/systemd/system/array-compression.service`:

```
1 [Unit]
2 Description=Array Compression Engine
3 After=network.target
4
5 [Service]
6 Type=simple
7 WorkingDirectory=/opt/arrayfolding
8 Environment=ARRAY_LICENSE_KEY=ARRAY-XXXXX-...
9 Environment=ARRAY_NATIVE_LIB=/opt/arrayfolding/libarray_core.so
10 ExecStart=/usr/bin/uvicorn server:app \
11     --host 0.0.0.0 --port 8200 --workers 2
12 Restart=always
13 RestartSec=5
14
15 [Install]
16 WantedBy=multi-user.target

1 systemctl enable array-compression
2 systemctl start array-compression
```

8.3 API Endpoints

Method	Endpoint	Description
POST	/compress	Compress a base64-encoded payload
POST	/decompress	Decompress a previously compressed payload
POST	/compress/vectors	Full pipeline: fold + quantize + compress
POST	/estimate	Estimate resource savings for a scenario
POST	/proxy	Transparent proxy: raw bytes in, compressed out
GET	/	Health check with cumulative statistics

9 Rocky Linux Container Support

9.1 Rocky Linux 9 — Validated

Rocky Linux 9.4 (glibc 2.34, Python 3.9) has been **validated** with 71/71 tests passing inside an LXC container. All SDK functions, BLSB bit-level operations, folding, compression, search index, and analysis functions work correctly.

Test environment: Proxmox LXC, Rocky Linux 9.4 (Blue Onyx), 2 vCPU, 4 GB RAM, glibc 2.34, Python 3.9.18, NumPy 2.0.2, GCC 11.4.1.

Validated results:

- **Folding:** 768D→16D, 384D→16D, 128D→16D, 1536D→32D, 64D→8D — all correct shapes, dtypes, deterministic
- **CPU:** 14–20× search speedup (768D vs 16D brute-force, 10K–100K vectors)
- **Memory:** 48× RAM reduction (614 MB → 12.8 MB for 100K vectors), 192× with uint16 quantization
- **BLSB:** 8/16/32-bit ratios correct, bridge quality Hamming distance correct, proof strategy selection correct
- **Storage:** 48× fold-only, 192× fold+quantize, 198× full pipeline (fold+quantize+zlib)
- **Compression:** AFLD compress/decompress roundtrip verified, `compress_vectors` pipeline verified
- **Search:** Index build, k-NN search, sorted distances, <2 ms latency for 10K vectors
- **Edge cases:** Small vectors (4D→2D), large/tiny magnitudes ($10^{12}/10^{-12}$), empty bytes, 1 MB random payloads

Rocky 9 container setup:

```
1 # Create on Proxmox (DHCP networking)
2 pct create 800 local:vztmpl/rockylinux-9-default_20240912_amd64.tar.xz \
3   --hostname rocky-afld --cores 2 --memory 4096 --swap 512 \
4   --rootfs local-lvm:20 --net0 name=eth0,bridge=vbr0,ip=dhcp \
5   --features nesting=1
6 pct start 800
7
8 # Push SDK files in, install pip + numpy offline if needed
9 pct push 800 arrayfolding-1.0.0-py3-none-any.whl /opt/arrayfolding/
10 pct push 800 libarray_core.so /opt/arrayfolding/
11 pct enter 800
12
13 # Inside the container
14 pip3 install /opt/arrayfolding/arrayfolding-1.0.0-py3-none-any.whl
15 export ARRAY_NATIVE_LIB=/opt/arrayfolding/libarray_core.so
```

Note: Rocky Linux minimal containers may not have outbound internet access depending on your network configuration. If `dnf` cannot reach mirrors, download pip, NumPy, and the SDK wheel on the Proxmox host and push them into the container with `pct push`.

9.2 Rocky Linux 8 — Under Testing

Rocky Linux 8 ships glibc 2.28, which is below the 2.31+ requirement for `libarray_core.so`. The pure-Python SDK fallback works, but native C acceleration has not been validated. **Do not deploy to Rocky 8 containers in production until testing is complete.**

If you require Rocky Linux 8 support, contact ck@advancedresearcharray.com.

10 Testing Checklist

Start with the Community license on a copy of your data in a non-production container. Test everything before production.

1. **Correctness.** Fold and unfold a sample. Compare distances between original and folded representations. Verify preservation rate $\geq 99\%$.
2. **Memory.** Monitor container RSS before and after folding. Confirm the expected reduction matches your data dimensions.
3. **CPU.** Benchmark search/comparison operations at original vs. folded dimensions. Measure wall-clock speedup.
4. **Storage.** Write folded + compressed data to disk. Verify you can decompress and reconstruct correctly.
5. **Container sizing.** After confirming folded memory requirements, reduce the container's RAM allocation. Run under load to verify no OOM kills.
6. **Persistence.** Restart the container. Verify folded/compressed data loads correctly from disk.
7. **Service health.** If running the compression API, verify the systemd service restarts correctly and the `GET / health` endpoint responds.

11 Disclaimer

This software is provided “as is” without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and non-infringement. In no event shall Advanced Research Array or its contributors be held liable for any data loss, corruption, service interruption, or any direct, indirect, incidental, special, or consequential damages arising from the use of this software. You are solely responsible for maintaining backups, testing in a non-production environment, and ensuring suitability for your use case before production deployment. By using this software you accept these terms.

References

- [1] Kilpatrick, C. (2025). “Database Dimensional Folding: 36 Quadrillion $\times$ Faster Searches Through 74D $\rightarrow$ 19D Projection.” Zenodo. <https://doi.org/10.5281/zenodo.18079591>
- [2] Kilpatrick, C. (2025). “Dimensional Folding Implementation Guide: From Theory to Production Deployment.” Zenodo. <https://doi.org/10.5281/zenodo.18357419>